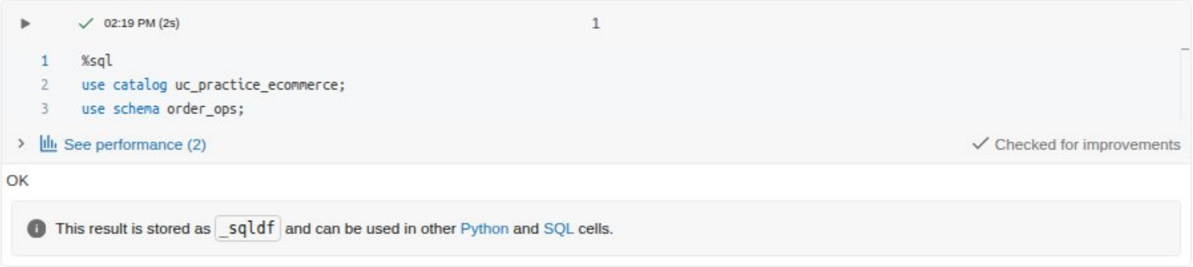
Soumiya_Ayyappan_w9_D2

Medium qn

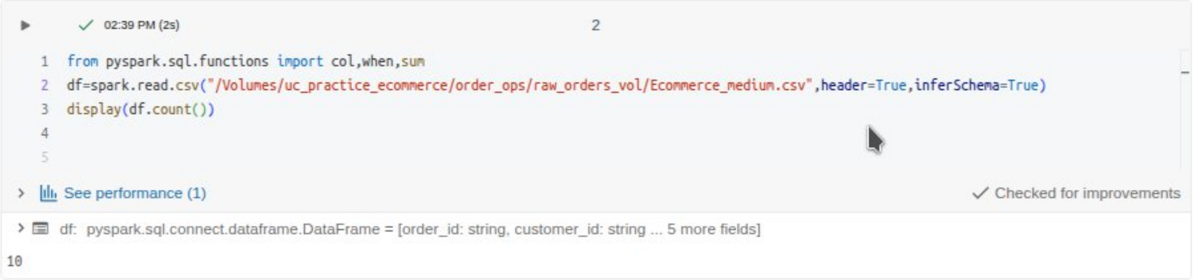
Step 1:



```
%sql
use catalog uc_practice_ecommerce;
use schema order_ops;
```

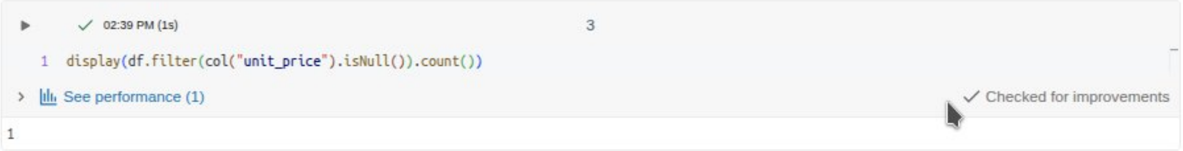
```
OK
```

This result is stored as `_sqldf` and can be used in other Python and SQL cells.



```
from pyspark.sql.functions import col,when,sum
df=spark.read.csv("/Volumes/uc_practice_ecommerce/order_ops/raw_orders_vol/Ecommerce_medium.csv",header=True,inferSchema=True)
display(df.count())
```

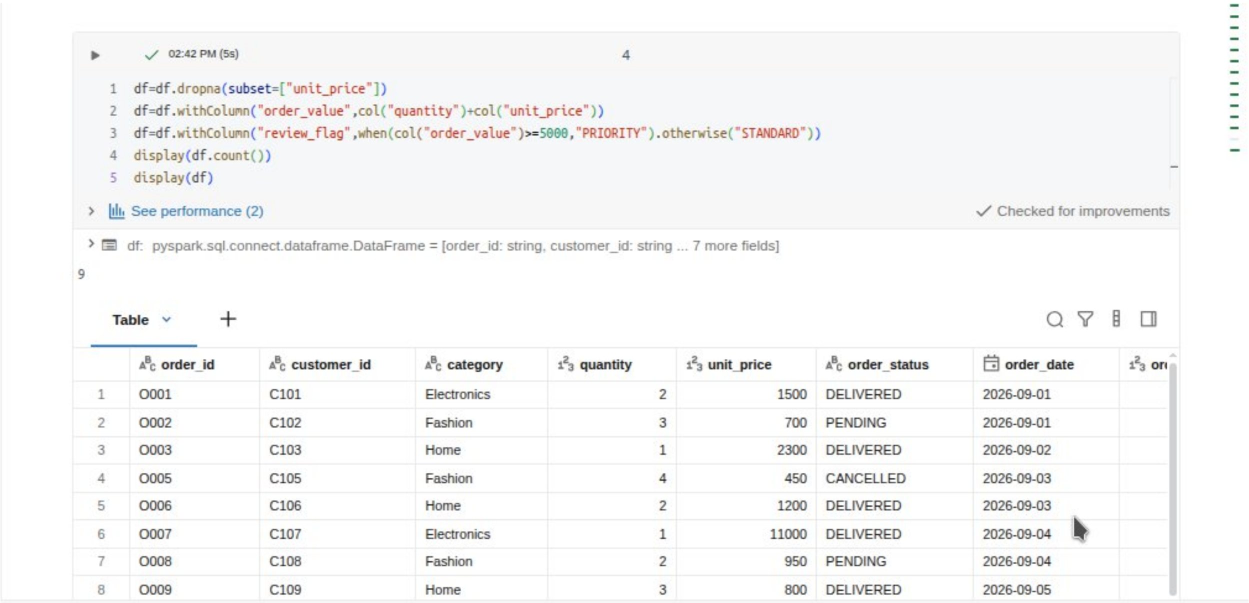
```
df:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, customer_id: string ... 5 more fields]
10
```



```
display(df.filter(col("unit_price").isNull()).count())
```

```
1
```

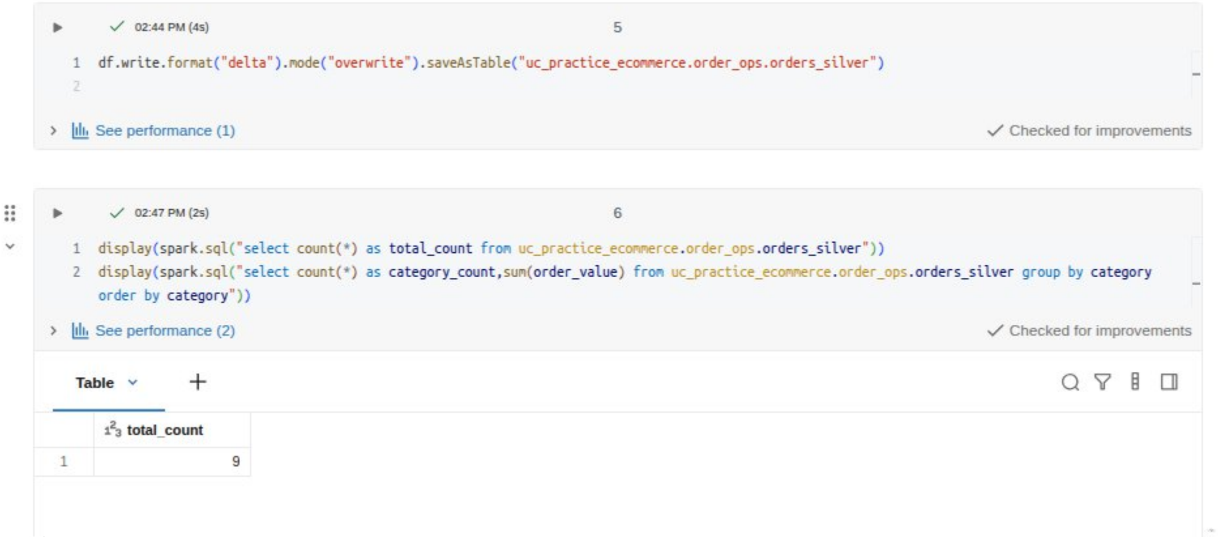
Step 2:



```
df=df.dropna(subset=["unit_price"])
df=df.withColumn("order_value",col("quantity")+col("unit_price"))
df=df.withColumn("review_flag",when(col("order_value")>=5000,"PRIORITY").otherwise("STANDARD"))
display(df.count())
display(df)
```

```
df:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, customer_id: string ... 7 more fields]
9
```

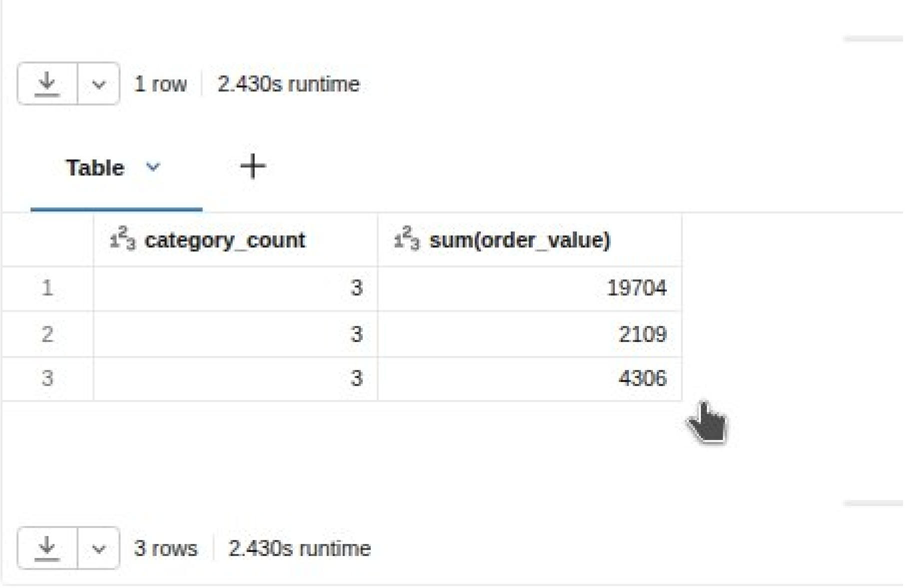
	order_id	customer_id	category	quantity	unit_price	order_status	order_date	or
1	O001	C101	Electronics	2	1500	DELIVERED	2026-09-01	
2	O002	C102	Fashion	3	700	PENDING	2026-09-01	
3	O003	C103	Home	1	2300	DELIVERED	2026-09-02	
4	O005	C105	Fashion	4	450	CANCELLED	2026-09-03	
5	O006	C106	Home	2	1200	DELIVERED	2026-09-03	
6	O007	C107	Electronics	1	11000	DELIVERED	2026-09-04	
7	O008	C108	Fashion	2	950	PENDING	2026-09-04	
8	O009	C109	Home	3	800	DELIVERED	2026-09-05	



```
df.write.format("delta").mode("overwrite").saveAsTable("uc_practice_ecommerce.order_ops.orders_silver")
```

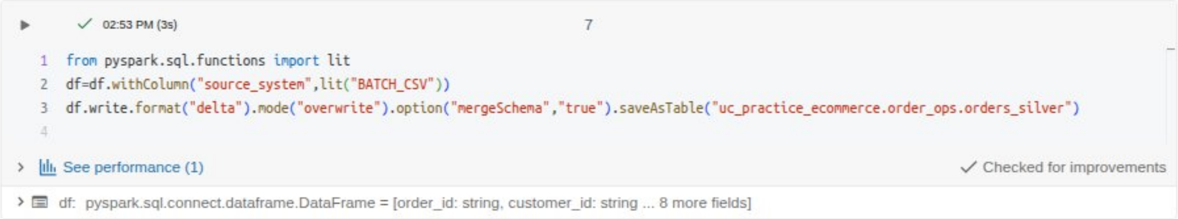
```
display(spark.sql("select count(*) as total_count from uc_practice_ecommerce.order_ops.orders_silver"))
display(spark.sql("select count(*) as category_count,sum(order_value) from uc_practice_ecommerce.order_ops.orders_silver group by category
order by category"))
```

	total_count
1	9



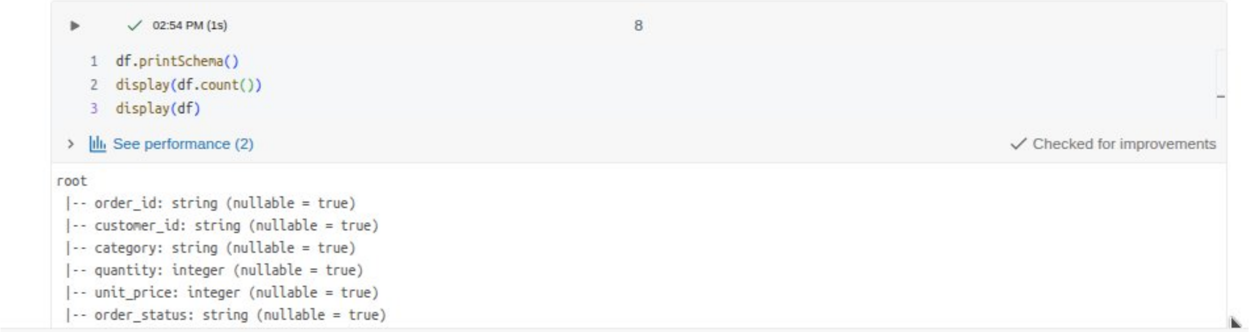
	category_count	sum(order_value)
1	3	19704
2	3	2109
3	3	4306

Step 3:



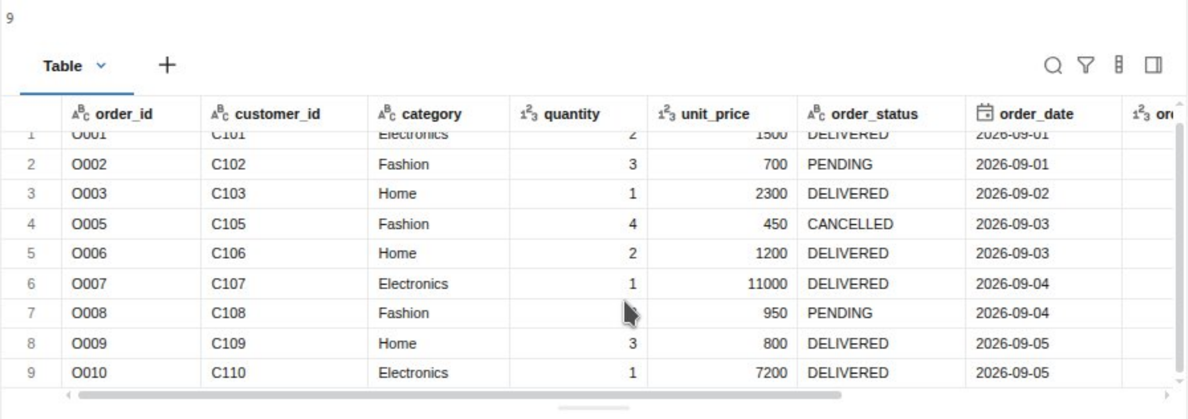
```
from pyspark.sql.functions import lit
df=df.withColumn("source_system",lit("BATCH_CSV"))
df.write.format("delta").mode("overwrite").option("mergeSchema","true").saveAsTable("uc_practice_ecommerce.order_ops.orders_silver")
```

```
df:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, customer_id: string ... 8 more fields]
```



```
df.printSchema()
display(df.count())
display(df)
```

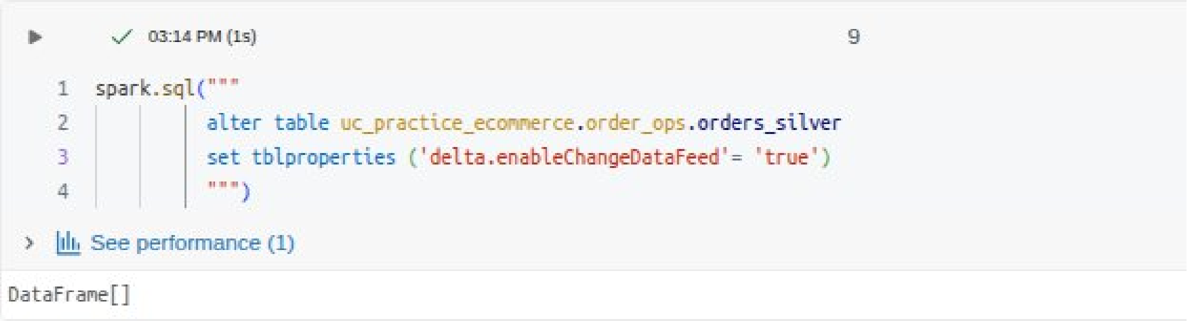
```
root
 |-- order_id: string (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- category: string (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- unit_price: integer (nullable = true)
 |-- order_status: string (nullable = true)
```



9

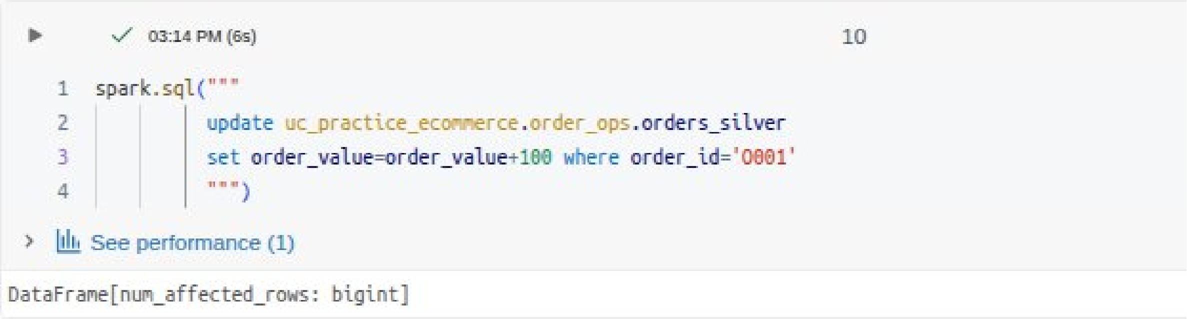
	order_id	customer_id	category	quantity	unit_price	order_status	order_date	or
1	O001	C101	Electronics	2	1500	DELIVERED	2026-09-01	
2	O002	C102	Fashion	3	700	PENDING	2026-09-01	
3	O003	C103	Home	1	2300	DELIVERED	2026-09-02	
4	O005	C105	Fashion	4	450	CANCELLED	2026-09-03	
5	O006	C106	Home	2	1200	DELIVERED	2026-09-03	
6	O007	C107	Electronics	1	11000	DELIVERED	2026-09-04	
7	O008	C108	Fashion		950	PENDING	2026-09-04	
8	O009	C109	Home	3	800	DELIVERED	2026-09-05	
9	O010	C110	Electronics	1	7200	DELIVERED	2026-09-05	

Step 4:



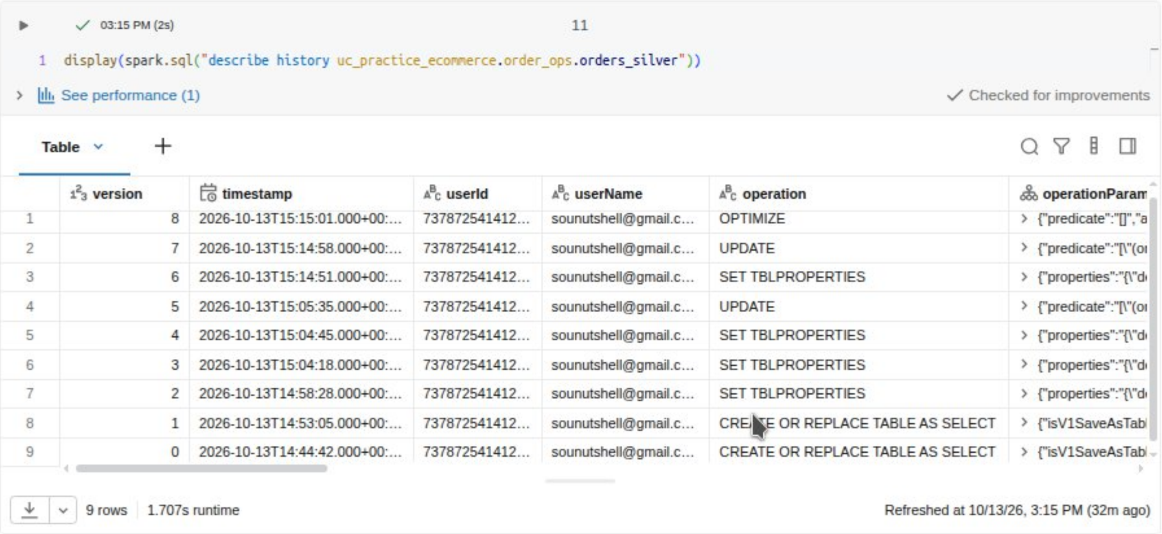
```
spark.sql("""
          alter table uc_practice_ecommerce.order_ops.orders_silver
          set tblproperties ('delta.enableChangeDataFeed'= 'true')
          """)
```

```
DataFrame[]
```



```
spark.sql("""
          update uc_practice_ecommerce.order_ops.orders_silver
          set order_value=order_value+100 where order_id='0001'
          """)
```

```
DataFrame[num_affected_rows: bigint]
```

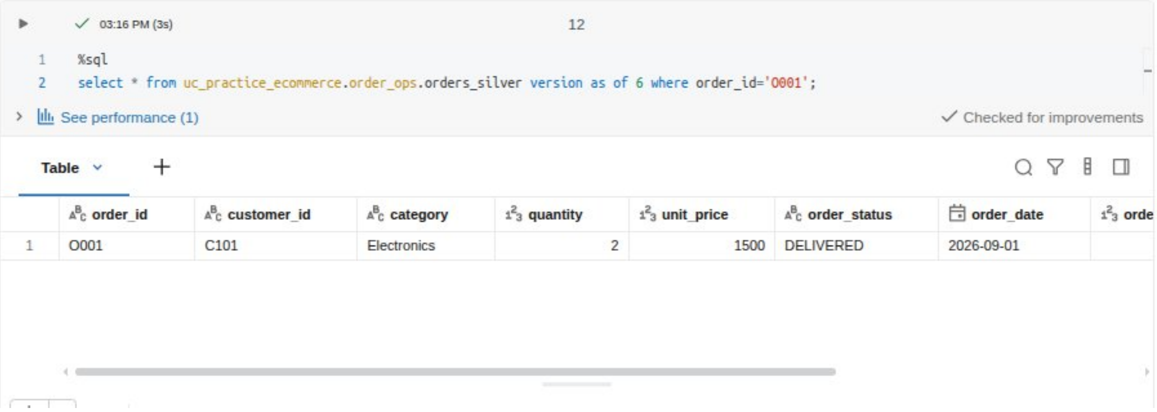


```
display(spark.sql("describe history uc_practice_ecommerce.order_ops.orders_silver"))
```

	version	timestamp	userId	userName	operation	operationParam
1	8	2026-10-13T15:15:01.000+00:...	737872541412...	sounutshell@gmail.c...	OPTIMIZE	{"predicate":"[]","a
2	7	2026-10-13T15:14:58.000+00:...	737872541412...	sounutshell@gmail.c...	UPDATE	{"predicate":"[\"(o
3	6	2026-10-13T15:14:51.000+00:...	737872541412...	sounutshell@gmail.c...	SET TBLPROPERTIES	{"properties":"{\"d
4	5	2026-10-13T15:05:35.000+00:...	737872541412...	sounutshell@gmail.c...	UPDATE	{"predicate":"[\"(o
5	4	2026-10-13T15:04:45.000+00:...	737872541412...	sounutshell@gmail.c...	SET TBLPROPERTIES	{"properties":"{\"d
6	3	2026-10-13T15:04:18.000+00:...	737872541412...	sounutshell@gmail.c...	SET TBLPROPERTIES	{"properties":"{\"d
7	2	2026-10-13T14:58:28.000+00:...	737872541412...	sounutshell@gmail.c...	SET TBLPROPERTIES	{"properties":"{\"d
8	1	2026-10-13T14:53:05.000+00:...	737872541412...	sounutshell@gmail.c...	CREATE OR REPLACE TABLE AS SELECT	{"isV1SaveAsTab
9	0	2026-10-13T14:44:42.000+00:...	737872541412...	sounutshell@gmail.c...	CREATE OR REPLACE TABLE AS SELECT	{"isV1SaveAsTab

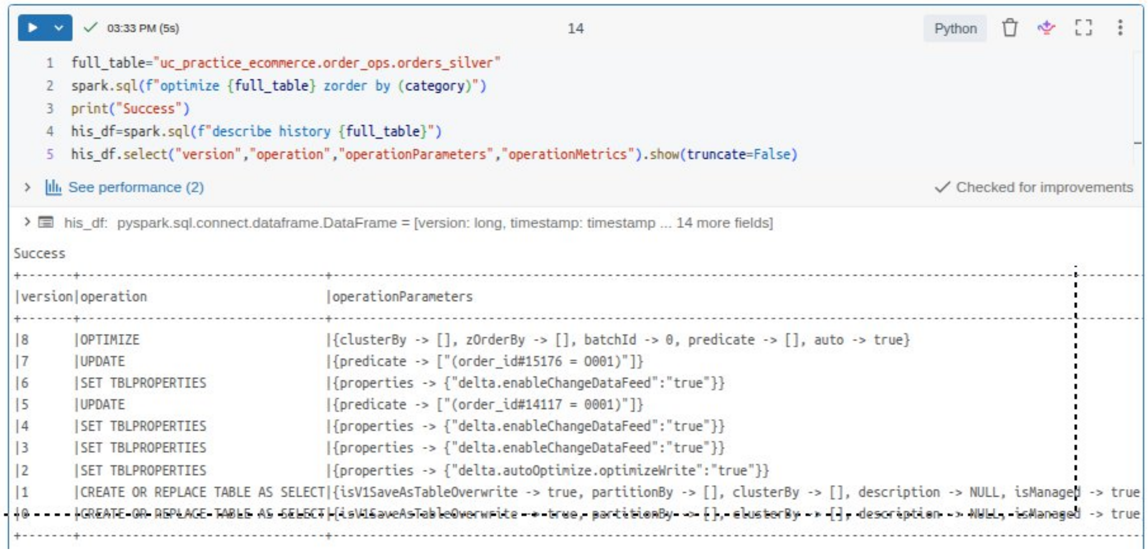
9 rows | 1.707s runtime

Refreshed at 10/13/26, 3:15 PM (32m ago)



```
%sql
select * from uc_practice_ecommerce.order_ops.orders_silver version as of 6 where order_id='O001';
```

	order_id	customer_id	category	quantity	unit_price	order_status	order_date	orde
1	O001	C101	Electronics	2	1500	DELIVERED	2026-09-01	



```
full_table="uc_practice_ecommerce.order_ops.orders_silver"
spark.sql(f"optimize {full_table} zorder by (category)")
print("Success")
his_df=spark.sql(f"describe history {full_table}")
his_df.select("version","operation","operationParameters","operationMetrics").show(truncate=False)
```

his_df: pyspark.sql.connect.dataframe.DataFrame = [version: long, timestamp: timestamp ... 14 more fields]

```
Success
+-------+--------------------------------+-------------------------------------------------------------------------------------------------------
|version|operation                       |operationParameters
+-------+--------------------------------+-------------------------------------------------------------------------------------------------------
|8      |OPTIMIZE                        |{clusterBy -> [], zOrderBy -> [], batchId -> 0, predicate -> [], auto -> true}
|7      |UPDATE                          |{predicate -> ["(order_id#15176 = O001)"]}
|6      |SET TBLPROPERTIES               |{properties -> {"delta.enableChangeDataFeed":"true"}}
|5      |UPDATE                          |{predicate -> ["(order_id#14117 = O001)"]}
|4      |SET TBLPROPERTIES               |{properties -> {"delta.enableChangeDataFeed":"true"}}
|3      |SET TBLPROPERTIES               |{properties -> {"delta.enableChangeDataFeed":"true"}}
|2      |SET TBLPROPERTIES               |{properties -> {"delta.autoOptimize.optimizeWrite":"true"}}
|1      |CREATE OR REPLACE TABLE AS SELECT|{isV1SaveAsTableOverwrite -> true, partitionBy -> [], clusterBy -> [], description -> NULL, isManaged -> true
|0      |CREATE OR REPLACE TABLE AS SELECT|{isV1SaveAsTableOverwrite -> true, partitionBy -> [], clusterBy -> [], description -> NULL, isManaged -> true
+-------+--------------------------------+-------------------------------------------------------------------------------------------------------
```